CLASE

El flujo de trabajo de Ansible



El flujo de trabajo

1

Escribir

Define el inventario (DÓNDE) y el playbook (QUÉ).

2

Verificar

Revisa la sintaxis y simula los cambios antes de aplicarlos (dry-run).

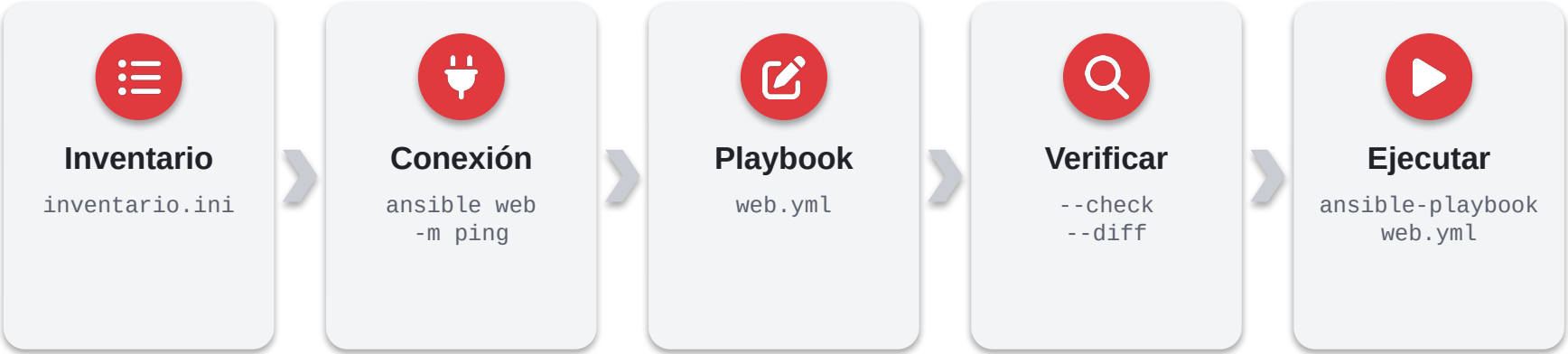
3

Ejecutar

Aplica la configuración en los servidores y revisa el resumen.

El flujo paso a paso

Cada etapa tiene su comando



Primero comprobamos que llegamos a los servidores, luego escribimos, simulamos y por último aplicamos.

El inventario

La lista de servidores que Ansible gestiona, organizados en grupos. Responde a la pregunta: ¿DÓNDE?

- Formato INI (el más simple) o YAML
- Los grupos van entre [corchetes]
- Grupo especial all: todos los hosts
- Variables por host: ansible_host, ansible_user
- Por defecto: /etc/ansible/hosts; lo normal es uno por proyecto

inventario.ini

```
[web]
web1 ansible_host=3.91.10.20
web2 ansible_host=3.91.10.21

[db]
db1 ansible_host=10.0.1.15

[all:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Explorar el inventario

TERMINAL

```
$ ansible-inventory -i inventario.ini --graph
@all:
  |--@ungrouped:
  |--@web:
  |  |--web1
  |  |--web2
  |--@db:
  |  |--db1
```

`--graph`

Árbol de grupos y hosts: ideal para comprobar el inventario.

`--list`

Todo el inventario en JSON, con sus variables.

`--host web1`

Solo las variables de un host concreto.

Probar la conexión

TERMINAL

```
$ ansible all -i inventario.ini -m ping
web1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
db1 | UNREACHABLE! => {
    "changed": false,
    "msg": "Failed to connect to the host
           via ssh: Connection timed out"
}
```

SUCCESS

Hay SSH y Python en el servidor: Ansible puede trabajar.

UNREACHABLE

- IP del inventario
- Usuario y clave SSH
- Security Group: puerto 22

El módulo ping de Ansible no es un ping ICMP: prueba la conexión SSH y Python.

Comandos ad-hoc

Una tarea rápida, sin escribir un playbook

```
ansible <grupo> -i <inventario> -m <módulo> -a "<argumentos>"
```

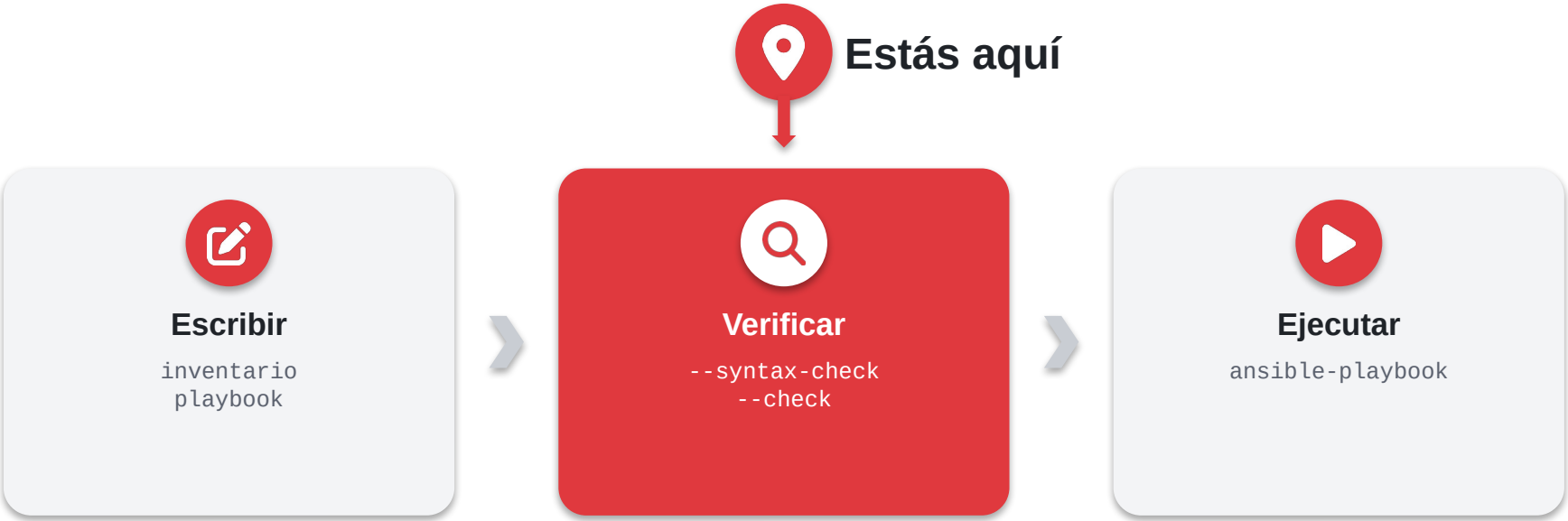
TERMINAL

```
# Consultar
$ ansible web -m command -a "uptime"
$ ansible web -m setup -a "filter=ansible_distribution*"
# Cambiar (con sudo: -b)
$ ansible web -b -m dnf -a "name=git state=present"
```

¿Cuándo usarlos?

- Consultas rápidas
- Diagnóstico
- Tareas únicas
- Para cambios repetibles → playbook

Flujo de trabajo



Verificar antes de ejecutar

--syntax-check

Revisa el YAML y la estructura del playbook. No se conecta a los servidores.

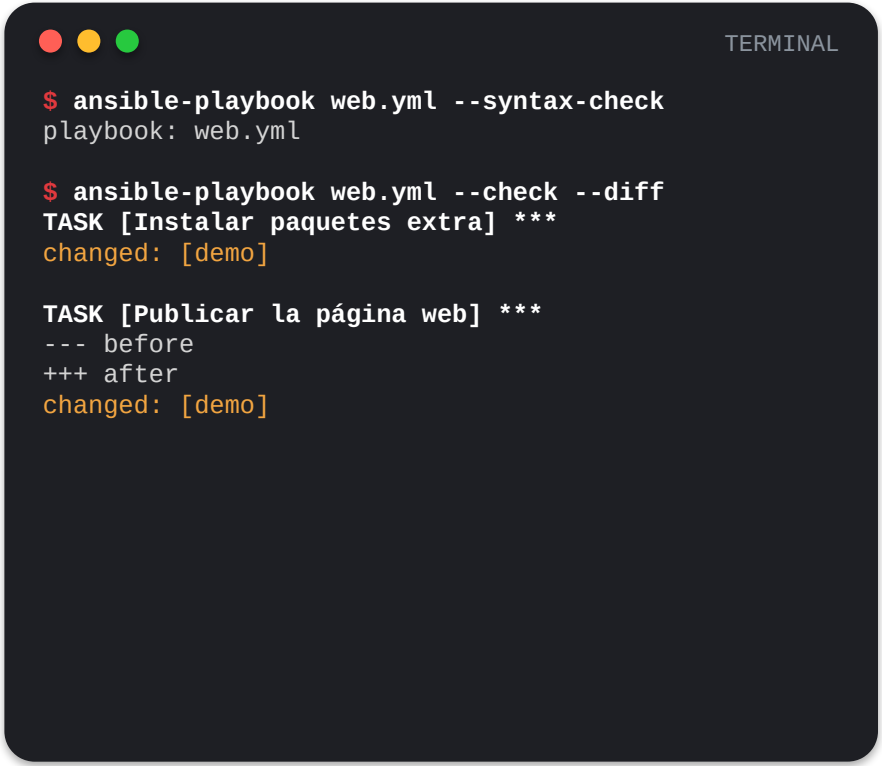
--check

Simulacro (dry-run): muestra qué cambiaría, sin cambiar nada.

--diff

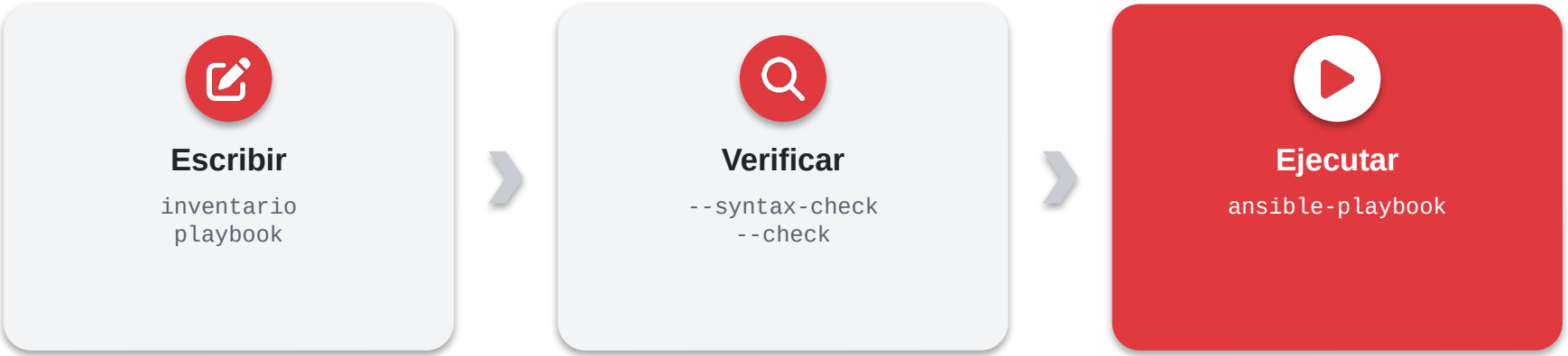
Muestra las líneas que cambiarían en los archivos.

Algunas tareas no se pueden simular: command y shell se saltan en modo --check.



Flujo de trabajo

Estás aquí



ansible-playbook en acción



Gathering Facts

Ansible recopila datos del servidor: SO, IP, memoria...

ok

La tarea se cumplió y no hizo falta cambiar nada.

changed

La tarea modificó algo en el servidor.

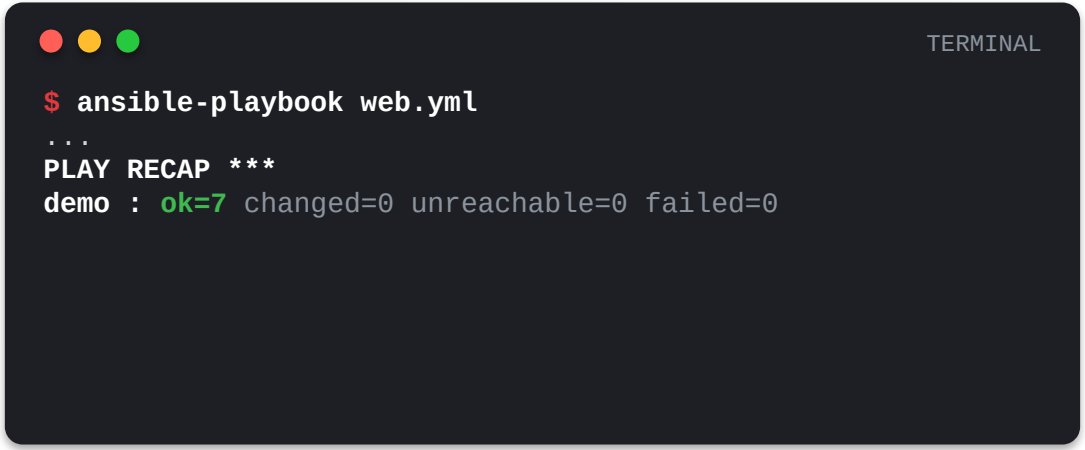
PLAY RECAP

Resumen final por host.

Leer el PLAY RECAP

Contador	Significado
`ok`	Tareas completadas con éxito (incluye las que hicieron cambios)
`changed`	Tareas que modificaron algo en el servidor
`unreachable`	No se pudo conectar al host (SSH, red, credenciales)
`failed`	La tarea falló: ese host se detiene; los demás siguen
`skipped`	Tarea saltada, por ejemplo porque no se cumplió un when:
`rescued / ignored`	Errores manejados (rescue) o ignorados (ignore_errors)

Idempotencia en acción



0

cambios en la segunda ejecución

- El servidor ya está en el estado deseado: Ansible comprueba y no toca nada.
- Es el equivalente al «No changes» de terraform plan.
- Si sale changed en la 2.ª ejecución, esa tarea no es idempotente: revísala (¿command o shell?).

Handlers: reaccionar a cambios

web.yml

```yaml
tasks:
  - name: Publicar la página web
    ansible.builtin.template:
      src: templates/index.html.j2
      dest: /var/www/html/index.html
    notify: Reiniciar Apache

handlers:
  - name: Reiniciar Apache
    ansible.builtin.service:
      name: httpd
      state: restarted
```

notify

La tarea avisa al handler por su nombre.

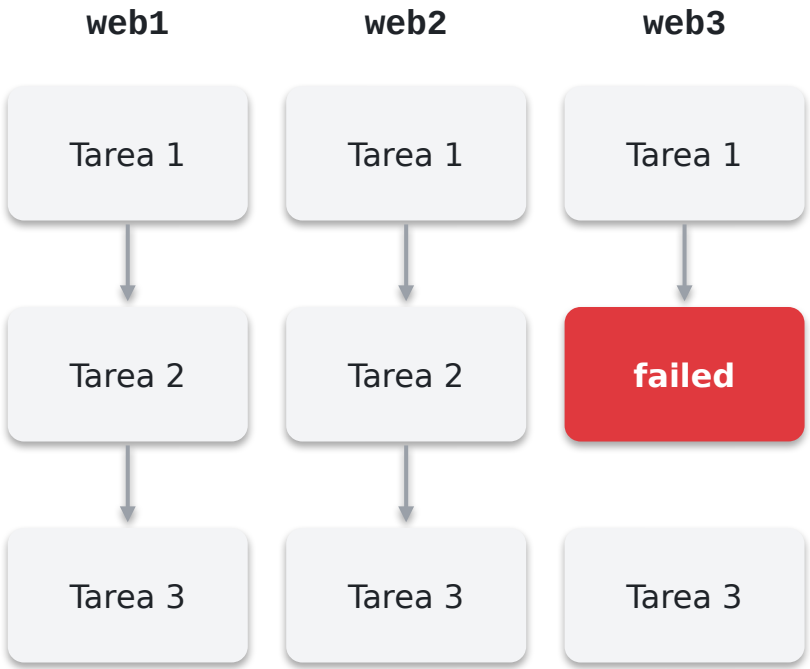
Solo si hubo cambios

Si la tarea sale ok, el handler no se ejecuta.

Al final del play

Se ejecuta una sola vez, aunque lo avisen varias tareas.

Orden de ejecución



Tareas en orden · hosts en paralelo

- **Tareas en orden:** la 2 no empieza hasta que la 1 termina en todos los hosts.
- **Hosts en paralelo:** 5 a la vez por defecto (forks). Se cambia con -f 10.
- **Si falla un host,** se detiene ese host; los demás continúan.
- **Sin rollback:** lo ya aplicado se queda aplicado.

¿Y cómo «destruyo»?

limpiar.yml

```yaml
- name: Desinstalar paquetes extra
  ansible.builtin.dnf:
    name: "{{ paquetes }}"
    state: absent
```

Ansible no tiene destroy: se declara el estado contrario con state: absent.

Ansible no guarda estado

- Si borras una tarea del playbook, lo que hizo sigue en el servidor.
- Terraform sí lo detecta y lo destruye porque tiene el .tfstate.
- Para quitar algo: escribe una tarea con state: absent.
- Para eliminar la VM entera: terraform destroy.

Buenas prácticas al ejecutar

1 Antes

- --syntax-check
- --check --diff
- Probar primero en dev
- --list-hosts: ¿a quién afecta?

2 Durante

- Leer qué sale changed
- Vigilar errores y avisos
- Limitar hosts con -l web1
- Paciencia con muchos hosts

3 Después

- Verificar el servicio
- Repetir: changed=0
- Commit del playbook en Git
- Documentar los cambios

RESUMEN

El flujo es escribir, verificar y ejecutar.

El inventario dice DÓNDE; el playbook dice QUÉ.

Prueba primero la conexión con ansible -m ping.

Verifica con --syntax-check y simula con --check --diff.

Lee el PLAY RECAP: ok, changed, failed, unreachable.

Una 2.ª ejecución con changed=0 confirma la idempotencia.